

AWS Developer Certification – Domain 1 Study Summary

Domain 1: Development with AWS Services — Task Statement 3: *Use Data Stores in Application Development*

Third in the Domain 1 series. Practice questions from the transcript appear as blockquotes, with a quick-reference answer key at the end.

Foundations to Review

- A solid data strategy underpins microservices: manage, act on, and react to data so you can make better decisions, respond faster, and turn data into insight.
- Dive deeper into AWS's data stores: **RDS, Aurora, DynamoDB, S3, EFS, EBS**, and more.
- Know which storage type fits which need — file server, block storage, or object storage — and which consistency model a given scenario calls for.

DynamoDB Global Secondary Indexes (GSIs)

- A GSI lets your application query a specific index across the entire base table — all partitions included.
- To create a table with one or more GSIs, use the `CreateTable` operation with the `GlobalSecondaryIndexes` parameter.
- A GSI **inherits its read/write capacity mode from the base table**.
- Queries/scans against a GSI consume capacity units **from the index, not the base table**.
- Queries against a GSI support **eventual consistency only**.

Consistency Models & Capacity Math

- S3 provides **strong read-after-write consistency**.
- **DynamoDB** supports both **eventually consistent** and **strongly consistent** reads.
- **Aurora and RDS** consistency models are flagged as a "dive deeper" item in this lesson — no specifics were given, so look these up separately.

Practice question — RCU capacity calculation: A DynamoDB table is provisioned with 10 RCU, and each item is 4 KB. How many strongly consistent and eventually consistent read requests can the table handle per second?

Method:

1. Provisioned RCU $\times$ 4 KB $\rightarrow 10 \times 4 \text{ KB} = 40$
2. Strongly consistent reads/sec = that result $\div$ 4 KB $\rightarrow 40 \div 4 \text{ KB} = \mathbf{10}$
3. Eventually consistent reads/sec = that result $\div$ 2 KB $\rightarrow 40 \div 2 \text{ KB} = \mathbf{20}$

Rule of thumb worth memorizing: for items ≤ 4 KB, strongly-consistent reads/sec $\approx$ provisioned RCU, and eventually-consistent reads/sec $\approx 2 \times$ provisioned RCU. (Also recall: 1 WCU = 1 KB/sec of write throughput.)

Storage Patterns: Ephemeral vs. Persistent

Ephemeral storage

- Services: **Lambda, EC2** (instance storage)
- Good fit for: ETL jobs, machine learning, data processing, downloading an S3 object in response to an S3 event, graphics processing

Persistent storage

- Services: **EFS, EBS, FSx, RDS, DynamoDB**
- Good fit for: web serving, content management, stateful microservices, machine learning
- The right storage choice depends on access method, access pattern, required throughput, frequency of access/update, and availability/durability needs.
- Well-architected systems typically combine multiple storage solutions rather than relying on one.
- In a microservices design, **each component should own its own data persistence layer**.

Data Lifecycle Management

- Lifecycle policies define where data is stored, for how long, and who can access it.
- **S3 and EFS** both support lifecycle policies (as do other storage options).
- **AWS Data Lifecycle Manager** automates the creation, retention, and deletion of **EBS snapshots**.

Data Management Goals & Caching for Performance

- Core data-management goals: **integrity, confidentiality, availability**, plus compliance and the safety of data in a disaster.
- To improve data retrieval performance, options include **in-memory engines/RAM-based caching, content delivery networks, and DNS-based approaches**.
- In distributed designs, consider a **dedicated caching layer** that runs independently from the data store's own cache and lifecycle.
- Best practices: validate the data being cached, use **TTLs** to expire stale data, and define an appropriate **RTO (recovery time objective)** and **RPO (recovery point objective)**.
- Dive deeper into caching strategies — pros, cons, and when to use **lazy loading** vs. **write-through**.

ETL & Data Serialization

- Think through serializing/deserializing data as you persist it, since your application likely handles multiple data types.
- Plan how to unify and load data into a persistent store for analysis and data-driven decisions.
- ETL tools to dive deeper into: **AWS Glue** and the **AWS Glue Data Catalog**, **EMR**, **Lambda**, and **Kinesis**.

Example: Serverless CRUD API

A common pattern for create/read/update/delete operations on a DynamoDB table:

- Build a serverless API with **Lambda + API Gateway**.
- Flow: the HTTP API is invoked → **API Gateway** routes the request to your **Lambda** function → Lambda interacts with the **DynamoDB** table → API Gateway returns the response.

DynamoDB Partition Key Best Practices

Know why a partition key matters and how to pick a good one:

- Use **high-cardinality attributes**.
- Use **composite attributes**.
- Cache popular items with **DAX**.
- For write-heavy workloads, add **random numbers/digits** to the key — this randomizing strategy improves write throughput, paired with a sharding approach for reading individual items.
- Choose a partition key strategy based on your data's ingestion and access patterns, aiming to minimize throttling.
- Know the difference between **queries** and **scans**, and when to use each.

Up Next

This lesson shifts from lecture into a **hands-on walkthrough question** — a format change rather than a move to a new task statement.

Quick-Reference Answer Key

#	Scenario / Question	Answer
1	Create a DynamoDB table with one or more GSIs	<code>CreateTable</code> operation with the <code>GlobalSecondaryIndexes</code> parameter
2	Consistency supported by GSI queries	Eventual consistency only
3	S3's consistency model	Strong read-after-write consistency
4	10 RCU, 4 KB items — strongly consistent reads/sec	10
5	10 RCU, 4 KB items — eventually consistent reads/sec	20
6	Automating EBS snapshot retention/creation/deletion	AWS Data Lifecycle Manager
7	Caching popular DynamoDB items	DAX